COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 6, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

Screenshots

diaGraph .h

```h
#ifndef DIAGRAPH_H
#define DIAGRAPH_H

#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;   // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head; // adjacency list as array of pointers

```

```
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        head = new adjNode*[N]();
        this->N = N;

        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }

    // Destructor (walks each list; the handout's delete[] head[i] is undefined behavior)
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* cur = head[i];
            while (cur != nullptr) {
                adjNode* nxt = cur->next;
                delete cur;
                cur = nxt;
            }
        }
        delete[] head;
    }
};
```

```

// print all adjacent vertices of given vertex
inline void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

#endif
```

main.cpp

```cpp
#include <iostream>
#include "diaGraph.h"


int main()
{
    //ILO A
    graphEdge edges[] = {
        // (x, y, w)
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph
    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph diagraph(edges, n, N);

    std::cout << "ILO A: Adjacency List" << std::endl;
    std::cout << "Graph adjacency list\n(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        display_AdjList(diagraph.head[i], i);
    }
    std::cout << std::endl;

```

output

```
ILO A: Adjacency List
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Screenshots	
graphTraversal1.h	

```
#ifndef GRAPHTRAVERSAL1_H
#define GRAPHTRAVERSAL1_H


#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

namespace DFS {

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
```

```
32      {
33          return this->weight > e.weight;
34      }
35  };
36
37  template <typename T>
38  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
39  {
40      for (auto i = 1; i < G.vertices(); i++)
41      {
42          os << i << ":\t";
43          auto edges = G.outgoing_edges(i);
44          for (auto &e : edges)
45              os << "{" << e.dest << ": " << e.weight << "}, ";
46          os << std::endl;
47      }
48      return os;
49  }
50
51  template <typename T>
52  class Graph
53  {
54  public:
55      // Initialize the graph with N vertices
56      Graph(size_t N) : V(N)
57      {
58      }
59
60      // Return number of vertices in the graph
61      auto vertices() const
62      {
```

```
63          return V;
64      }
65
66      // Return all edges in the graph
67      auto &edges() const
68      {
69          return edge_list;
70      }
71
72      void add_edge(Edge<T> &&e)
73      {
74          // Check if the source and destination vertices are within range
75          if (e.src >= 1 && e.src <= V &&
76              e.dest >= 1 && e.dest <= V)
77              edge_list.emplace_back(e);
78          else
79              std::cerr << "Vertex out of bounds" << std::endl;
80      }
81
82      // Returns all outgoing edges from vertex v
83      auto outgoing_edges(size_t v) const
84      {
85          std::vector<Edge<T>> edges_from_v;
86          for (auto &e : edge_list)
87          {
88              if (e.src == v)
89                  edges_from_v.emplace_back(e);
90          }
91          return edges_from_v;
92      }
93
```

```
94      // Overloads the << operator so a graph be written directly to a stream
95      // Can be used as std::cout << obj << std::endl;
96      template <typename U>
97      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
98
99  private:
00      size_t V; // Stores number of vertices in graph
01      std::vector<Edge<T>> edge_list;
02  };
03
04  template <typename T>
05  auto depth_first_search(const Graph<T> &G, size_t dest)
06  {
07      std::stack<size_t> stack;
08      std::vector<size_t> visit_order;
09      std::set<size_t> visited;
10      stack.push(1); // Assume that DFS always starts from vertex ID 1
11      while (!stack.empty())
12      {
13          auto current_vertex = stack.top();
14          stack.pop();
15          // If the current vertex hasn't been visited in the past
16          if (visited.find(current_vertex) == visited.end())
17          {
18              visited.insert(current_vertex);
19              visit_order.push_back(current_vertex);
20              for (auto e : G.outgoing_edges(current_vertex))
21              {
22                  // If the vertex hasn't been visited, insert it in the stack.
23                  if (visited.find(e.dest) == visited.end())
24                  {
```

```
                        stack.push(e.dest);
                    }
                }
            }
        }
        return visit_order;
    }

    template <typename T>
    auto create_reference_graph()
    {
        Graph<T> G(9);

        std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
        edges[1] = {{2, 0}, {5, 0}};
        edges[2] = {{1, 0}, {5, 0}, {4, 0}};
        edges[3] = {{4, 0}, {7, 0}};
        edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
        edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
        edges[6] = {{4, 0}, {7, 0}, {8, 0}};
        edges[7] = {{3, 0}, {6, 0}};
        edges[8] = {{4, 0}, {5, 0}, {6, 0}};

        for (auto &i : edges)
            for (auto &j : i.second)
                G.add_edge(Edge<T>{i.first, j.first, j.second});

        return G;
    }

    template <typename T>
```

```
156 void test_DFS()
157 {
158     // Create an instance of and print the graph
159     auto G = create_reference_graph<unsigned>();
160     std::cout << G << std::endl;
161 
162     // Run DFS starting from vertex ID 1 and print the order
163     // in which vertices are visited.
164     std::cout << "DFS Order of vertices: " << std::endl;
165     auto dfs_visit_order = depth_first_search(G, 1);
166     for (auto v : dfs_visit_order)
167         std::cout << v << std::endl;
168 }
169 
170 }
171 
172 #endif
```

main.cpp

```
32      //ILO B
33      using T = unsigned;
34
35      std::cout << "ILO B.1: Depth-First Search (DFS)" << std::endl;
36      DFS::test_DFS<T>();
37      std::cout << std::endl;
```

output

```
ILO B.1: Depth-First Search (DFS)
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

Screenshots

graphTraversal2.h

```
#ifndef GRAPHTRAVERSAL2_H
#define GRAPHTRAVERSAL2_H


#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

namespace BFS {

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
```

```
31          return this->weight > e.weight;
32      }
33  };
34
35  template <typename T>
36  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
37  {
38      for (auto i = 1; i < G.vertices(); i++)
39      {
40          os << i << ":\t";
41          auto edges = G.outgoing_edges(i);
42          for (auto &e : edges)
43              os << "{" << e.dest << ": " << e.weight << "}, ";
44          os << std::endl;
45      }
46      return os; // added: missing in the handout
47  }
48
49  template <typename T>
50  class Graph
51  {
52  public:
53      Graph(size_t N) : V(N) {}
54
55      auto vertices() const
56      {
57          return V;
58      }
59
60      auto &edges() const
61      {
```

```
62          return edge_list;
63      }
64
65      void add_edge(Edge<T> &&e)
66      {
67          if (e.src >= 1 && e.src <= V &&
68              e.dest >= 1 && e.dest <= V)
69              edge_list.emplace_back(e);
70          else
71              std::cerr << "Vertex out of bounds" << std::endl;
72      }
73
74      auto outgoing_edges(size_t v) const
75      {
76          std::vector<Edge<T>> edges_from_v;
77          for (auto &e : edge_list)
78          {
79              if (e.src == v)
80              {
81                  edges_from_v.emplace_back(e);
82              }
83          }
84          return edges_from_v;
85      }
86
87      template <typename U> // changed from T: reusing T shadows the class parameter
88      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```
89
90  private:
91      size_t V;
92      std::vector<Edge<T>> edge_list;
93  };
94
95  template <typename T>
96  auto create_reference_graph()
97  {
98      Graph<T> G(9);
99
100     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
101     edges[1] = {{2, 2}, {5, 3}};
102     edges[2] = {{1, 2}, {5, 5}, {4, 1}};
103     edges[3] = {{4, 2}, {7, 3}};
104     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
105     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
106     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
107     edges[7] = {{3, 3}, {6, 4}};
108     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
109
```

```
110     for (auto &i : edges)
111         for (auto &j : i.second)
112             G.add_edge(Edge<T>{i.first, j.first, j.second});
113 
114     return G;
115 }
116 
117 template <typename T>
118 auto breadth_first_search(const Graph<T> &G, size_t dest)
119 {
120     std::queue<size_t> queue;
121     std::vector<size_t> visit_order;
122     std::set<size_t> visited;
123     queue.push(1); // Assume that BFS always starts from vertex ID
124 
125     while (!queue.empty())
126     {
127         auto current_vertex = queue.front();
128         queue.pop();
129         // If the current vertex hasn't been visited in the past
130         if (visited.find(current_vertex) == visited.end())
131         {
132             visited.insert(current_vertex);
133             visit_order.push_back(current_vertex);
134             for (auto e : G.outgoing_edges(current_vertex))
135                 queue.push(e.dest);
136         }
137     }
138     return visit_order;
139 }
140 
```

```
141  template <typename T>
142  void test_BFS()
143  {
144      // Create an instance of and print the graph
145      auto G = create_reference_graph<unsigned>();
146      std::cout << G << std::endl;
147
148      // Run BFS starting from vertex ID 1 and print the order
149      // in which vertices are visited.
150      std::cout << "BFS Order of vertices: " << std::endl;
151      auto bfs_visit_order = breadth_first_search(G, 1);
152      for (auto v : bfs_visit_order)
153          std::cout << v << std::endl;
154  }
155
156  }
157
158  #endif
```

main.cpp

```
39      std::cout << "ILO B.2: Breadth-First Search (BFS)" << std::endl;
40      BFS::test_BFS<T>();
```

output

```
ILO B.2: Breadth-First Search (BFS)
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Observation:

The procedure showed that the graph programs were able to run successfully and produce the expected outputs. The adjacency list displayed the connections between vertices, while the DFS and BFS programs produced their respective vertex traversal orders. The outputs also showed that the graph was traversed without getting stuck or repeatedly processing the same vertices. Overall, the procedures and outputs were completed as expected.

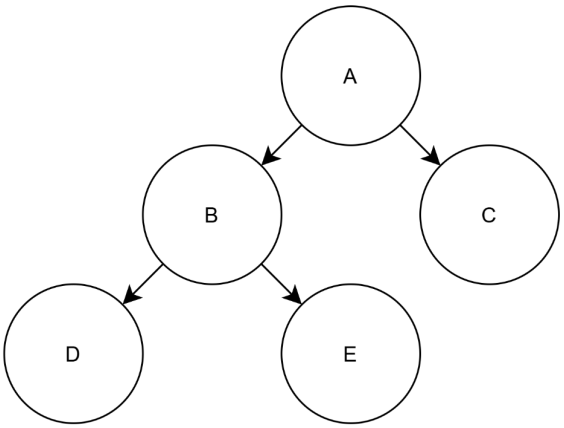
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
Depth First Search would be the most helpful algorithm. It starts from one vertex, explores as far as possible along one path, then backtracks to the previous vertex and explores other unvisited

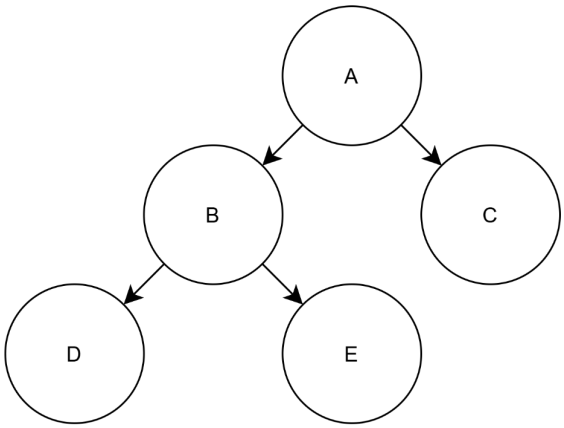
2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

DFS in a Graph



Visit: A → B → D → E → C

DFS in a Tree



Pre-order: A → B → D → E → C

PSEUDOCODE	
DFS	DFS(vertex): mark vertex as visited for each adjacent vertex: if not visited: DFS(adjacent vertex)
Tree Preorder Traversal	Preorder(node): if node is not NULL: visit node Preorder(node.left) Preorder(node.right)

CODE

```
    void DFS(Node* node) {
    if (node == NULL)
        return;

    cout << node->data << " ";
    DFS(node->left);
    DFS(node->right);
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
Queue is the data structure used to implement Breadth First Search (BFS). It follows the First-In, First-Out (FIFO) principle.

4. How many times can a node be visited in the BFS?
In BFS, a node is generally visited only once. A visited array or similar mechanism is used to prevent the same node from being processed repeatedly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly and ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used for grammar correction, while ChatGPT was used to explain concepts from the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.